A New Algorithm for Applying Sequences of Affine Transformations in Quantum Circuits

Anish Giri
Department of Computer Science
Vanderbilt University
Nashville, Tennessee 37212, USA
Email: anish.giri@vanderbilt.edu

David Hyde, *Senior Member, IEEE*
Department of Computer Science
Vanderbilt University
Nashville, Tennessee 37212, USA
Email: david.hyde.1@vanderbilt.edu

Kálmán Varga
Department of Physics and Astronomy
Vanderbilt University
Nashville, Tennessee 37240, USA
Email: kalman.varga@vanderbilt.edu

arXiv:2412.09714v2 [quant-ph] 17 Apr 2025

***Abstract*—This paper introduces a novel technique for applying sequences of affine transformations in quantum circuits. Utilizing Hadamard-supported conditional initialization and block encoding, the proposed method systematically applies sequential affine transformations while preserving state normalization. The combinatorial amplitude patterns of the quantum states generated with our method enable applications in fields such as financial risk assessment, where we demonstrate computing portfolio returns using combinatorial sum of amplitudes, and discrete signal processing, where we demonstrate manipulation of the Fourier coefficients of a signal.**

***Index Terms*—Affine transformations, quantum circuits, state initialization**

I. Introduction

Affine transformations combine linear transformations and translations as $A\Psi + B$, where $\Psi \in \mathbb{R}^N$ is the input vector, $A \in \mathbb{R}^{N\times N}$ is the linear transformation matrix, and $B \in \mathbb{R}^N$ is the translation vector. Affine transformations are foundational in machine learning [1] and iterative methods [2] for tasks like image processing, regression, and data analysis, where the relative positions of elements need to be retained while allowing for scaling, rotations, and shear transformations.

In quantum computing, the implementation of affine transformations presents a challenge because quantum operations are inherently unitary (time reversible), i.e., $U^\dagger U = I$, preserving the norm of quantum states. However, a simple affine transformation will alter the norm of a state vector, which would suggest that one cannot implement affine transformations with quantum circuits. However, recent works have demonstrated multiple approaches for quantum implementations of affine transformations. For instance, one approach proposes encoding pixel positions and employing controlled-NOT (CNOT) gates and adders for operations like scaling, translation, and scrambling, thereby enabling efficient manipulation of quantum-encoded images [3]. Another approach leverages the synthesis of reversible circuits from invertible matrices, using CNOT gates to represent linear transformations and combining them with simple classical elements like NOT and XOR gates for affine shifts [4]. From a more structural perspective, polyhedral models have been utilized to optimize the scheduling and allocation of gates in circuits that apply affine transformations, effectively handling execution dependencies through loop transformations and affine maps [5]. Polyhedral abstractions combined with integer linear programming (ILP) techniques have enabled the derivation of optimal qubit indexing schemes, minimizing gate counts and circuit depths while respecting device constraints [6]. Similarly, quantum circuits that realize affine mappings over finite fields F_{2^n} have been explored using modular arithmetic and normal basis representations, yielding efficient and scalable arithmetic circuits [7].

While these works illustrate the potential of implementing affine transformations in quantum settings, there remains a need for a more scalable algorithm that can apply a series of affine transformations. One traditional approach, which borrows from classical fields like computer graphics [8], uses homogeneous coordinates to define an augmented state vector on which the affine transformation $f(\Psi) = A\Psi + B$ can be performed using a single matrix-vector multiplication in a higher-dimensional space. Specifically, we may define

$$\tilde{\Psi} = \frac{1}{\sqrt{2}} \begin{bmatrix} \Psi \\ 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} \in \mathbb{R}^{2N},$$

and then the augmented matrix

$$\tilde{A} = \begin{bmatrix} A^{N\times N} & 0^{N\times(N-1)} & B^{N\times 1} \\ 0^{N\times N} & & I^{N\times N} \end{bmatrix} \in \mathbb{R}^{(2N)\times(2N)}.$$

With this representation, the affine transformation can be expressed as a purely linear transformation in the augmented space:

$$\tilde{f}(\tilde{\Psi}) = \tilde{A}\tilde{\Psi}.$$

However, this matrix is not unitary. To correct this, we can perform a standard block encoding trick, which embeds the $(2N)\times(2N)$ normalized non-unitary matrix $\tilde{A}/\alpha$ into a larger $(4N)\times(4N)$ unitary matrix

$$U(\tilde{A}) = \begin{pmatrix} \tilde{A} & \sqrt{I - \tilde{A}\tilde{A}^\dagger} \\ \sqrt{I - \tilde{A}^\dagger\tilde{A}} & -\tilde{A}^\dagger \end{pmatrix},$$

This work was supported by the National Science Foundation under Grant Number 2442853.

where $\alpha > 0$ is a normalization factor to ensure that $\|\tilde{A}/\alpha\| \leq 1$ [9]–[11]. The encoded unitary $U(\tilde{A})$ can be applied conditionally to the desired number of qubits to produce the affine-transformed amplitudes in the quantum state. In this way, block encoding can be combined with linear transformations in augmented spaces to achieve affine transformations of vectors or fields.

However, constructing a block-encoding procedure is challenging and becomes increasingly computationally demanding as the matrix size grows. Employing an augmented affine matrix using block encoding leads to a fourfold increase in matrix size, which can necessitate a greater number of qubits and makes decomposing the operator into a product of basis gates more challenging [12]–[14] (at this time, factoring a unitary into an approximate product of basis gates remains challenging even for a small number of qubits [15]). These considerations motivate the search for more scalable and efficient strategies to perform affine transformations within the memory of a quantum computer.

In this paper, we introduce a scalable method for applying sequences of affine transformations on a field localized within a sub-state of a quantum system. Our approach uses block encoding of A instead of $\tilde{A}$, which leads to a matrix size of $4N^2$ instead of $16N^2$, and employs Hadamard-supported conditional initialization of the translation terms. This method not only facilitates execution of sequences of affine transformations but also generates combinatorial patterns of additive and subtractive translation elements, effectively accumulating these amplitudes across various sub-states. This holds possibilities for further quantum computing applications such as quantum combinatorial optimization, signal processing, and quantum machine learning [16]–[18]. With recent advancements in quantum computing algorithms for solving differential equations [19]–[21], combinatorial optimization through quantum gradient descent [22], quantum approaches to solving solid mechanics problems [23], and plasma simulations [24], an algorithm of this nature can potentially offer efficient iterative methods for addressing these complex problems.

The paper is organized as follows. In the next section, we present a simple method for element-wise addition and subtraction in an $n+1$-qubit system. Following this, a step-by-step method for applying sequences of affine transformations is introduced. The result is generalized to any number of affine transformations for any number of qubits. Finally, two potential applications are demonstrated: one in financial risk assessment and the other in discrete signal processing.

II. Hadamard-supported Element-wise Addition and Subtraction

Several approaches have been developed to achieve arithmetic operations within quantum states using Fourier-arithmetic techniques [25]–[28]. However, these methods are either not suitable or applicable for performing arithmetic directly on a large number of field values within a single quantum state while allowing the flexibility to apply extra unitary operations within the quantum circuit. Instead, we build our idea on using amplitudes of quantum states to represent the field values and perform arithmetic on those amplitudes.

A significant challenge in quantum systems is the inability to directly add or subtract bias terms from amplitudes while maintaining normalization. For instance, consider the states:

$$\Psi_A = a_1 |00\rangle + a_2 |01\rangle + a_3 |10\rangle + a_4 |11\rangle ,$$

$$\Psi_B = b_1 |00\rangle + b_2 |01\rangle + b_3 |10\rangle + b_4 |11\rangle .$$

If we simply added the two sets of amplitudes, we could end up with a state whose total probability (the L_2 norm of the vector of probability amplitudes) exceeds 1:

$$\begin{aligned}\Psi_{(A+B)} = &(a_1 + b_1) |00\rangle + (a_2 + b_2) |01\rangle \\ &+ (a_3 + b_3) |10\rangle + (a_4 + b_4) |11\rangle .\end{aligned}$$

However, it is possible to create a quantum state where $a_i + b_i$ is accumulated over states, scaled by a known factor, using Hadamard-supported conditional initialization as demonstrated in Fig. 1. We note that conditional initialization, in general, is often available in quantum computing software development kits, such as Qiskit; and a similar approach in which Ψ_A and Ψ_B are sub-states of a quantum state and are conditionally implemented between two Hadamard gates on the ancillary qubit is demonstrated in [29]. Conditional initialization prepares one sub-state when the ancilla is $|0\rangle$ and another sub-state when the ancilla is $|1\rangle$. Suppose the two quantum states are given by:

$$\Psi_A = \sum_{i=1}^{N} a_i |i\rangle , \quad \Psi_B = \sum_{i=1}^{N} b_i |i\rangle ,$$

where $N = 2^n$. The system is prepared with an $(n+1)$-qubit register in the state:

$$\Psi_0 = |0\rangle \otimes |0\rangle^{\otimes n} .$$

Here, the first qubit acts as an ancillary qubit, while the remaining n qubits are the target register that will store the quantum states. A Hadamard gate H is applied to the ancilla, creating an equal superposition of its computational basis states:

$$\Psi_1 = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle^{\otimes n} .$$

This superposition ensures that subsequent initialization of the target qubits depends conditionally on the ancilla's state. Conditional initialization is then applied to the last n qubits. When the ancilla is in $|0\rangle$, the target qubits are prepared in the state Ψ_B. When the ancilla is in $|1\rangle$, the target qubits are prepared in the state Ψ_A. The resulting state becomes:

$$\Psi_2 = \frac{1}{\sqrt{2}} \left(|0\rangle \otimes \Psi_B + |1\rangle \otimes \Psi_A\right) .$$

Finally, a second Hadamard gate is applied to the ancilla, mixing the states and producing:

$$\Psi_3 = \frac{1}{2} \left[|0\rangle \otimes (\Psi_A + \Psi_B) + |1\rangle \otimes (\Psi_A - \Psi_B)\right] .$$

Expanding this, the amplitudes of Ψ_A and Ψ_B are combined:

$$\Psi_3 = \frac{1}{2}\sum_{i=1}^{N}\Big[(a_i + b_i)\,|0\rangle\,|i\rangle + (a_i - b_i)\,|1\rangle\,|i\rangle\Big].$$

This final state encodes both the element-wise sum $a_i + b_i$ and difference $a_i - b_i$, scaled to preserve normalization. The final state can yield just the scaled sum or just the scaled difference by measuring the ancilla qubit in the $\{|0\rangle, |1\rangle\}$ basis and then projecting the state onto either the sum ($|0\rangle$) or the difference ($|1\rangle$) component.

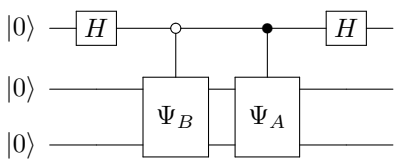


Fig. 1. Hadamard-supported element-wise addition and subtraction for a three-qubit system, where the first qubit is ancillary and the remaining two qubits are conditionally initialized with Ψ_A and Ψ_B.

We note that implementations of conditional state initialization are enabled by recent advancements in quantum state initialization algorithms, particularly the divide-and-conquer and top-down approaches. The divide-and-conquer algorithm achieves an exponential reduction in circuit depth, scaling from $O(N)$ in traditional top-down methods to $O(\log_2^2(N))$ [30], [31]. However, this improvement comes at the cost of increased qubit usage, requiring $O(N)$ ancillary qubits compared to $O(\log_2(N))$ qubits in top-down methods [32]. Depending on the requirements and resource constraints, either state initialization technique can be applied conditionally.

III. Sequential Affine Transformations

In this section, we discuss how to apply sequential affine transformations to an input field that is accumulated over a sub-state of a quantum system. Given a discrete field $\Psi \in \mathbb{R}^N$, the objective is to achieve the following series of transformations:

$$\Psi^N_{\text{final}} = A_k^{N\times N}\big(\cdots A_2^{N\times N}\big(A_1^{N\times N}\Psi + B_1^N\big) + B_2^N\big)\cdots + B_k^N$$

Here, $A_j^{N\times N}$ is an $N \times N$ linear transformation matrix, and B_j^N is a column translation vector applied at each step j.

To achieve the final amplitudes for this transformation, accumulated over a sub-state in a quantum circuit, the circuit width must be strategically expanded, along with constructing a unitary version of each A_j. Additionally, each B_j must be resized appropriately to match the expanded dimensions of the associated unitary version of A_j. At each step j, the process involves Hadamard-supported element-wise addition and subtraction, followed by conditional initialization of the resulting state with an additional ancillary qubit. These steps ensure the sequential application of the affine transformations within the quantum circuit. The complete sequence of operations is outlined as follows:

Step 1: Amplitude Initialization Starting with an s-qubit system initialized in the state $|0\rangle^{\otimes s}$, we encode a given (user-supplied) normalized vector $\Psi = [x_1, x_2, \ldots, x_N]^T \in \mathbb{R}^N$ into the amplitudes of the last n qubits ($n \leq s$) of the system, where $N = 2^n$[1]. This yields an initialized state

$$\Psi_0 = |0\rangle^{\otimes(s-n)} \otimes \Big(\sum_{i=1}^{N} x_i\,|i\rangle\Big).$$

Step 2: Apply Unitary Transformation To apply the first transformation matrix $A_1^{N\times N}$, we construct a unitary $U_1 \in \mathbb{R}^{(2N)\times(2N)}$ using block encoding. This unitary acts on $n+1$ qubits and transforms the state such that the amplitude corresponding to $|i\rangle$ becomes $x_i^{U_1}$. The resulting quantum state is given by:

$$U_1\Psi_0 = \quad |0\rangle^{\otimes(s-n-1)} \otimes \Big(\sum_{i=1}^{2N} x_i^{U_1}\,|i\rangle\Big).$$

Note that in the above we define $x_i = 0$ for $i = N+1, \ldots, 2N$ (though by the action of U_1, the corresponding $x_i^{U_1}$ may be non-zero). At this point, we have effectively applied $A_1^{N\times N}$ to our system by using an extra qubit, and the remaining step is to add B_1^N.

Step 3: Hadamard-supported Add/Sub with B_1^N First, we zero-pad B_1^N (which we also assume is normalized) to $B_1 \in \mathbb{R}^{2N}$ (i.e., extending to $n+1$ qubits). We apply Hadamard-supported addition and subtraction of $B_1 = [\omega_1, \omega_2, \ldots, \omega_{2N}]^T$ on the existing quantum states to perform amplitude arithmetic. As suggested by Section II, this yields the state

$$\begin{aligned}\Psi_1 = \quad & |0\rangle^{\otimes(s-n-2)} \otimes \frac{1}{2}\Bigg(\sum_{i=1}^{2N}\Big(x_i^{U_1} + \omega_i\Big)\,|0\rangle \otimes |i\rangle \\ & + \sum_{i=1}^{2N}\Big(x_i^{U_1} - \omega_i\Big)\,|1\rangle \otimes |i\rangle\Bigg).\end{aligned}$$

After this step, the first affine transformation is achieved with its result—stored in the probability amplitudes of the first N $|0i\rangle$ states, in the last $n+2$ qubits—scaled by a known factor of 1/2 (which can be undone classically by the user after measurement). For clarity, we note that the $|0\rangle \otimes |i\rangle$ and $|1\rangle \otimes |i\rangle$ terms are inside their respective sums, not outside the sums.

Next Iteration: To establish the general pattern for applying sequences of affine transformations, we show what applying a second affine transformation then involves. For the second affine transformation, we block encode A_2 into the unitary U_2.

[1]Both our method and the technique on the first page of the manuscript assume a normalized input state. It is possible to extend either technique to handle non-normalized input states with the addition of ancillary qubits and with larger matrices.

This unitary is applied to the last $n+3$ qubits of the quantum state as follows:

$$U_2\Psi_1 = \quad |0\rangle^{\otimes(s-n-3)} \otimes \frac{1}{2}\left(\sum_{i=1}^{4N}\left((x_i^{U_1}+\omega_i)^{U_2}\right)|0i\rangle + \sum_{i=1}^{4N}\left((x_i^{U_1}-\omega_i)^{U_2}\right)|1i\rangle\right).$$

We zero-pad the x_i and ω_i as needed to make the above equation valid.

We then perform Hadamard-supported addition and subtraction using the normalized vector B_2^N. We wish to construct a B_2 analogous to B_1, but here we encounter a subtle detail where care is required. We form the zero-padded vector $B_2 = [\beta_1, \beta_2, \ldots, \beta_{8N}]^T$ ($\beta_i = 0$ for $i = N+1, \ldots, 8N$). We then rescale the vector so that the first $4N$ entries become $\beta_i/2$, and the remaining entries are chosen via an appropriate scheme to ensure the state remains normalized[2]. We label this rescaled vector $\tilde{B}_2$.

With this $\tilde{B}_2$ in mind, applying Hadamard-supported additional and subtraction with $\tilde{B}_2$ modifies the amplitudes to yield a final state:

$$\begin{aligned}\Psi_2 = \quad & |0\rangle^{\otimes(s-n-4)} \otimes \frac{1}{2}\Bigg[\sum_{i=1}^{8N}\left(\frac{(x_i^{U_1}+\omega_i)^{U_2}}{2}+\tilde{\beta}_i\right)|0\rangle\otimes|0i\rangle \\ & + \sum_{i=1}^{8N}\left(\frac{((x_i^{U_1}-\omega_i)^{U_2}}{2}+\tilde{\beta}_i\right)|0\rangle\otimes|1i\rangle \\ & + \sum_{i=1}^{8N}\left(\frac{((x_i^{U_1}+\omega_i)^{U_2}}{2}-\tilde{\beta}_i\right)|1\rangle\otimes|0i\rangle \\ & + \sum_{i=1}^{8N}\left(\frac{((x_i^{U_1}-\omega_i)^{U_2}}{2}-\tilde{\beta}_i\right)|1\rangle\otimes|1i\rangle\Bigg],\end{aligned}$$

where $\tilde{\beta}_i$ is the ith entry of $\tilde{B}_2$. Note that for the first $2N$ terms in the first sum in the above expression, $\tilde{\beta}_i = \beta_i/2$. So if one measures the first N states $|0\rangle \otimes |0i\rangle$, $i = 1, \ldots, N$, one can account for a known constant factor of $1/4$ that can be pulled out front, and one is thereby able to recover the result of the first and second affine transformation being sequentially applied. We note that the remaining $7N$ states may hold useless values, which is irrelevant since we do not need to measure those states.

IV. Generalized Affine Transformation Steps

The generalized procedure for k affine transformation steps is described by a sequence of block-encoded unitaries and Hadamard-supported addition/subtraction operations. At each step j (with $1 \le j \le k$), we have a linear transformation $A_j^{N\times N}$ embedded into a unitary U_j, which doubles the state dimension from $2^{j-1}N$ to 2^jN, and a translation (or bias) vector B_j^N that is resized and normalized to form an associated $\tilde{B}_j$. Then a Hadamard-supported operation is applied that splits the state into two branches, labeled by a binary variable $b_j \in \{0,1\}$, thus giving the amplitudes

$$\Phi^{(j)}_{(b_1,\ldots,b_j),i} = \frac{1}{2}\left[\left(\Phi^{(j-1)}_{(b_1,\ldots,b_{j-1}),i}\right)^{U_j} + (-1)^{b_j}\,\tilde{\beta}_i^{(j)}\right].$$

.

Thus, if we start with the initial state $\Phi_i^{(0)} = x_i$ ($i = 1, \ldots, N$, with $N = 2^n$) and have our associated normalized input state

$$\Psi = |0\rangle^{\otimes(s-n)} \otimes \sum_{i=1}^{N} x_i\,|i\rangle\,,$$

then after k steps, the full quantum state is given by

$$\Psi_{\text{final}} = |0\rangle^{\otimes(s-n-2k)} \otimes \Phi_{\text{aff}},$$

$$\Phi_{\text{aff}} = \frac{1}{2^k}\sum_{(b_1,\ldots,b_k)\in\{0,1\}^k}\sum_{i=1}^{2^kN}\Phi^{(k)}_{(b_1,\ldots,b_k),\,i}\,|b_1b_2\cdots b_k,\,i\rangle\,.$$

The result of the sequence of affine transformations (scaled by $1/2^k$, which is trivial to undo classically after measurement) can then be extracted as described in the prior section.

Algorithm 1 Sequential Quantum Affine Transformations

```
function SEQ_AFF_TRANSFORM(Ψ, {A_1, ..., A_K}, {B_1, ..., B_K}, n, s)
    current_state ← INITIALIZE_AMPLITUDES(Ψ, n, s)
    for j = 1 to K do
        s ← s + 1
        U_j ← BLOCK_ENCODE(A_j, 2^(n+j))
        current_state ← APPLY_UNITARY(U_j, current_state)
        s ← s + 1
        B̃_j ← RESCALE(B_j, 2^(n+j))
        current_state ← HADAMARD_ADD_SUB(current_state, B̃_j)
    end for
    return current_state
end function
```

V. Gate counts comparison

We compare our new approach with the existing block encoded augmented matrix approach. In the former (our) approach, after the system is initialized with some amplitude, a 4×4 matrix A is directly embedded into an 8×8 unitary U_1 using a square-root-based embedding and polar decomposition, and this unitary is then incorporated into a 4-qubit circuit via a controlled operation combined with state preparation to produce the Hadamard supported add/sub operation. Meanwhile, in the latter approach (see Section I), an augmented matrix $\tilde{A}$ is constructed by concatenating A with a 4-dimensional vector B along with suitable zero and identity blocks, and the resulting 16×16 operator U_2 is similarly embedded into a 4-qubit circuit.

The gate counts for the decomposed circuits are summarized in Table I. Notably, our new approach uses substantially (32%) fewer total gates than the block encoded augmented matrix method, though it introduces a higher number of multi-qubit gates, which may affect implementation cost on hardware. We note that since our algorithm is more scalable than the method described in Section I, we would expect to see even larger performance advantages for systems of more qubits.

[2]For instance, the second $4N$ entries ($i = 4N+1, \ldots, 8N$), rather than being zero, could be set to $\sqrt{1-\beta_{i-4N}^2/4}$. Other strategies are possible.

TABLE I
GATE COUNT COMPARISON BETWEEN THE TWO METHODS IMPLEMENTED IN QISKIT FOR A 4-QUBIT SYSTEM.

Approach	Single-Qubit	Multi-Qubit	Total
Our New Approach	93	120	213
Block Encoded Aug. Matrix	209	104	313

VI. APPLICATIONS

A. Financial Risk Assessment

The efficient computation of portfolio returns and their associated risks is a fundamental problem in computational finance, and quantum computing offers potential solutions to this challenge [33]–[35]. Here, we propose a simple knapsack variant approach to portfolio return using our sequential affine transformation approach in a quantum circuit.

Consider a normalized portfolio list $P = [a_1, a_2, \ldots, a_N]$ with $N = 2^m$ and sorted so that $a_i \geq a_{i+1}$. We define $\Psi = [a_1, a_2]$ and $B_1 = [a_3, a_4]$, and for each $k \geq 2$, we let B_k be the next 2^k assets of P. A portfolio return R incorporates sign variables $S \in \{-1, 1\}$ multiplying each asset, and we prescribe this via factors $(-1)^{i_r}$, where $i \in \{0,1\}^N$ and r is an index $1 \leq r \leq N$. R comprises multiple sums over different assets grouped by their importance and can be efficiently encoded into a quantum state using a sequential affine technique procedure. (And in this case, each A_i is an identity matrix.)

We begin with a one-qubit system initialized to the state Ψ and perform a Hadamard-supported add/subtract operation with B_1. We then expand the system by adding one qubit, forming a two-qubit system, and apply the same operation with B_2. This process is repeated iteratively: at each step k, we add one additional qubit to form a $(k+1)$-qubit system and perform the Hadamard-supported add/subtract operation using B_k. At the k-th step, the system contains 2^{k+1} amplitude values corresponding to the computational basis states of the $(k+1)$-qubit system.

For the given portfolio P, the final output of the successive Hadamard-supported add/subtract steps can be expressed in closed form by associating each output element with an m-tuple of bits $i = (i_0, i_1, \ldots, i_{m-1})$ as:

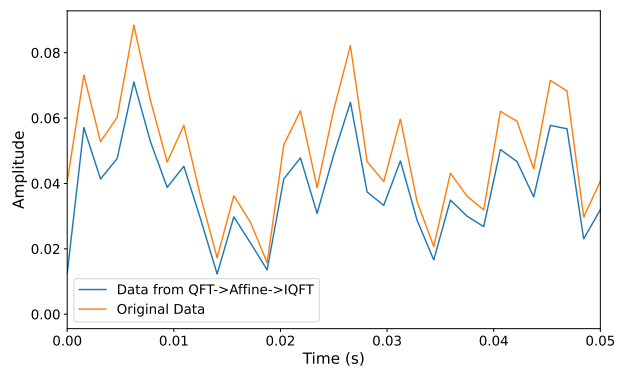
$$F(i_0, i_1, \ldots, i_{m-1}) = (-1)^{i_0}\, a_{1+i_0} + \sum_{r=1}^{m-1} (-1)^{i_r}\, a_{2^r+1+\sum_{s=0}^{r-1} 2^{r-1-s}\, i_s}\,.$$

Each Hadamard-supported add/sub scales the amplitude by 1/2, which means the F is scaled by $1/2^k$ after k such operations. These scaled amplitudes present in the quantum system can be estimated by repeatedly preparing the quantum state, measuring in the computational basis, and recording the frequency of each bitstring outcome $(i_0, i_1, \ldots, i_{m-1})$, which corresponds to the squared magnitude $|F(i_0, i_1, \ldots, i_{m-1})|^2$ of the amplitude.

As described, this procedure simply prepares a combinatorial number of states that add/subtract assets in P in the probability amplitudes of a quantum state. Although not all 2^N possible states are represented, the way we organize the states ensures larger assets are more likely to appear in the states. By making multiple measurements, one could observe which states have highest probability and thus have largest F, for instance; other financial conclusions could be drawn by practitioners, and this algorithm could be extended to perform techniques like amplitude amplification (to bring out the state with largest F) depending on the desired financial application.

Recent advancements in quantum portfolio optimization have shown the potential to significantly accelerate such computations using quantum parallelism [36], [37]. In portfolio management, this enables efficient computation of weighted combinations of asset returns using a constant number of Hadamard-supported conditional initializations. This approach could be extended to solve a variety of optimization problems, including other knapsack variants, offering substantial computational advantages [35], [38].

B. Discrete Signal Processing



(a) Illustrates the original signal and the result of our quantum procedure (QFT → Affine Transformation → Inverse QFT).

(b) Schematic of a quantum circuit in Qiskit implementing these steps on a 10-qubit quantum system.

Fig. 2. Discrete signal processing using our quantum affine transformation algorithm. Following a QFT operation on an input signal, a single affine transformation modifies a signal in Fourier space, and an inverse QFT transforms the altered signal back to the time domain.

Discrete signal processing techniques are pivotal in enhancing signals, suppressing noise, filtering image and audio data, and enabling efficient data compression [39], [40]. These transformations facilitate precise control of signal properties

in both the time and frequency domains. Classically, the discrete Fourier transform (DFT) of the signal vector $X = [x_1, x_2, \ldots, x_N]$ is given by:

$$X_k = \sum_{n=1}^{N} x_n \, e^{-j\frac{2\pi}{N}(k-1)(n-1)}, \quad k = 1, 2, \ldots, N,$$

where N is the signal length, and X_k represents the complex Fourier coefficients in the frequency domain [41].

To manipulate the signal in the frequency domain, an affine transformation is applied to the Fourier coefficients:

$$\tilde{X}_k = a \cdot X_k + b,$$

where a is a scaling factor for amplitude modification, and b introduces a constant bias. The modified coefficients are transformed back into the time domain using the Inverse Discrete Fourier Transform (IDFT):

$$\tilde{x}_n = \frac{1}{N} \sum_{k=1}^{N} \tilde{X}_k \, e^{j\frac{2\pi}{N}(k-1)(n-1)}, \quad n = 1, 2, \ldots, N.$$

We conduct an example of performing an affine transformation of a signal in frequency space using the quantum Fourier transform (QFT) and our proposed method. Given a discrete signal

$$x_n = \sum_{i=1}^{L} A_i \sin(2\pi f_i t_n),$$

where t_n represents uniform sampling in time, f_i are the frequencies, and A_i are the corresponding amplitudes of the sinusoidal components, we encode the A_i as amplitudes of a quantum state. We then perform QFT on the signal [42], followed by an affine transformation in the frequency domain (using our method), and then finally an inverse QFT. The results of this experiment on a random such input signal are shown in Fig. 2.

This methodology enables precise control over signal characteristics, such as selectively enhancing or suppressing components for applications in audio processing, image filtering, and data compression. Of course, multiple layers of affine transformations could be performed in the frequency domain such that different filters are applied in each transformation.

VII. Conclusion

In summary, this work presents a novel routine for applying sequences of affine transformations to the probability amplitudes of an input quantum state. By representing affine transformations in an augmented vector space and using block encoding and Hadamard-supported conditional initialization, the proposed method efficiently generates combinatorial amplitude patterns while maintaining state normalization. Potential utility of the approach is demonstrated through preliminary experiments in financial risk assessment and discrete signal processing. As quantum computing continues to advance, kernels like sequential affine transformations will be increasingly valuable for tackling complex computational tasks spanning optimization, machine learning, signal processing, and beyond. Future research could extend our approach to multidimensional transformations and explore its potential in quantum machine learning and combinatorial optimization applications.

References

[1] J. Cao, K. Zhang, H. Yong, X. Lai, B. Chen, and Z. Lin, "Extreme learning machine with affine transformation inputs in an activation function," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, no. 7, pp. 2093–2107, 2019.

[2] H. Li, X. Huang, J. Huang, and S. Zhang, "Feature matching with affine-function transformation models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 12, pp. 2407–2422, 2014.

[3] H. Liang, X. Tao, and N. Zhou, "Quantum image encryption based on generalized affine transform and logistic map," *Quantum Information Processing*, vol. 15, no. 7, pp. 2701–2724, 2016.

[4] X. Zeng, G. Yang, X. Song, M. A. Perkowski, and G. Chen, "Detecting affine equivalence of boolean functions and circuit transformation," *The Computer Journal*, vol. 66, pp. 2220–2229, 07 2022.

[5] M. Kong, "On the impact of affine loop transformations in qubit allocation," *ACM Transactions on Quantum Computing*, vol. 2, Sept. 2021.

[6] B. Gerard and M. Kong, "Exploring affine abstractions for qubit mapping," in *2021 IEEE/ACM Second International Workshop on Quantum Computing Software (QCS)*, pp. 43–54, 2021.

[7] M. Khan and A. Rasheed, "A fast quantum image encryption algorithm based on affine transform and fractional-order Lorenz-like chaotic dynamical system," *Quantum Information Processing*, vol. 21, no. 134, 2022.

[8] S. Marschner and P. Shirley, *Fundamentals of Computer Graphics*. CRC Press, 4th ed., 2016.

[9] P. Rall, "Quantum algorithms for estimating physical quantities using block encodings," *Phys. Rev. A*, vol. 102, p. 022408, Aug 2020.

[10] B. D. Clader, A. M. Dalzell, N. Stamatopoulos, G. Salton, M. Berta, and W. J. Zeng, "Quantum resources required to block-encode a matrix of classical data," *IEEE Transactions on Quantum Engineering*, vol. 3, pp. 1–23, 2022.

[11] A. Gilyén, Y. Su, G. H. Low, and N. Wiebe, "Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics," in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2019, (New York, NY, USA), pp. 193–204, Association for Computing Machinery, 2019.

[12] D. Camps, L. Lin, R. V. Beeumen, and C. Yang, "Explicit quantum circuits for block encodings of certain sparse matrices," 2023.

[13] H. Li, H. Ni, and L. Ying, "On efficient quantum block encoding of pseudo-differential operators," *Quantum*, vol. 7, p. 1031, June 2023.

[14] D. Camps and R. Van Beeumen, "Approximate quantum circuit synthesis using block encodings," *Phys. Rev. A*, vol. 102, p. 052411, Nov 2020.

[15] Y. Huang, B. E. Grossman-Ponemon, and D. A. B. Hyde, "Automated synthesis of quantum algorithms via classical numerical techniques," 2024.

[16] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, pp. 195–202, Sep 2017.

[17] I. Kerenidis and A. Prakash, "Quantum recommendation systems," 2016.

[18] M. Schuld and F. Petruccione, *Supervised Learning with Quantum Computers*. Quantum Science and Technology, Springer International Publishing, 2018.

[19] Y. Cao, A. Papageorgiou, I. Petras, J. Traub, and S. Kais, "Quantum algorithm and circuit design solving the poisson equation," *New Journal of Physics*, vol. 15, p. 013021, jan 2013.

[20] A. M. Childs, J.-P. Liu, and A. Ostrander, "High-precision quantum algorithms for partial differential equations," *Quantum*, vol. 5, p. 574, Nov. 2021.

[21] N. Linden, A. Montanaro, and C. Shao, "Quantum vs. classical algorithms for solving the heat equation," 2020.

[22] X. Yi, J.-C. Huo, Y.-P. Gao, L. Fan, R. Zhang, and C. Cao, "Iterative quantum algorithm for combinatorial optimization based on quantum gradient descent," *Results in Physics*, vol. 56, p. 107204, 2024.

[23] B. Liu, M. Ortiz, and F. Cirak, "Towards quantum computational mechanics," *Computer Methods in Applied Mechanics and Engineering*, vol. 432, p. 117403, Dec. 2024.

[24] I. Y. Dodin and E. A. Startsev, "On applications of quantum computing to plasma simulations," *Physics of Plasmas*, vol. 28, p. 092101, 09 2021.

[25] L. Ruiz-Perez and J. C. Garcia-Escartin, "Quantum arithmetic with the quantum Fourier transform," *Quantum Information Processing*, vol. 16, no. 6, p. 152, 2017.

[26] T. Häner, M. Roetteler, and K. M. Svore, "Optimizing quantum circuits for arithmetic," 2018.

[27] M. K. Thomsen, R. Glück, and H. B. Axelsen, "Reversible arithmetic logic unit for quantum arithmetic," *Journal of Physics A: Mathematical and Theoretical*, vol. 43, no. 38, p. 382002, 2010.

[28] R. Seidel, N. Tcholtchev, S. Bock, C. K.-U. Becker, and M. Hauswirth, "Efficient floating point arithmetic for quantum computers," *IEEE Access*, vol. 10, pp. 72400–72415, 2022.

[29] P. Wittek, *Quantum Machine Learning: What Quantum Computing Means to Data Mining*. Academic Press, 2014.

[30] I. F. Araujo, D. K. Park, F. Petruccione, *et al.*, "A divide-and-conquer algorithm for quantum state preparation," *Scientific Reports*, vol. 11, p. 6329, 2021.

[31] D. Ventura and T. Martinez, "Initializing the amplitude distribution of a quantum state," *Foundations of Physics Letters*, vol. 12, pp. 547–559, 1999.

[32] I. F. Araujo, D. K. Park, T. B. Ludermir, *et al.*, "Configurable sublinear circuits for quantum state preparation," *Quantum Information Processing*, vol. 22, p. 123, 2023.

[33] S. Woerner and D. J. Egger, "Quantum risk analysis," *npj Quantum Information*, vol. 5, p. 15, 2019.

[34] P. Rebentrost and S. Lloyd, "Quantum computational finance: Pricing derivatives," *Physical Review A*, vol. 98, no. 2, p. 022321, 2018.

[35] D. J. Egger, S. Woerner, and I. Tavernelli, "Quantum computing for finance: State-of-the-art and future prospects," *npj Quantum Information*, vol. 6, p. 1, 2020.

[36] R. Orus, S. Mugel, and E. Lizaso, "Quantum approaches to portfolio optimization," *arXiv preprint arXiv:1811.12344*, 2018.

[37] G. Rosenberg and et al., "Quantum machine learning for credit scoring and risk management," *PLOS One*, vol. 14, no. 8, p. e0221304, 2019.

[38] A. Bouland and S. Gharibian, "On the complexity and applications of quantum portfolio optimization," *Quantum*, vol. 4, p. 334, 2020.

[39] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*. Pearson Education, 2009.

[40] A. Papoulis, "Applications of discrete signal processing in modern technology," *Journal of Applied Signal Processing*, vol. 10, no. 3, pp. 210–223, 2012.

[41] R. N. Bracewell, "A tutorial on Fourier transform, discrete Fourier transform, and fast Fourier transform," *Proceedings of the IEEE*, vol. 82, no. 3, pp. 414–428, 1994.

[42] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition ed., 2010.